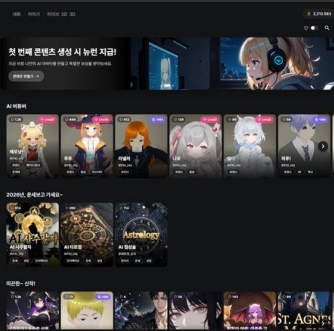


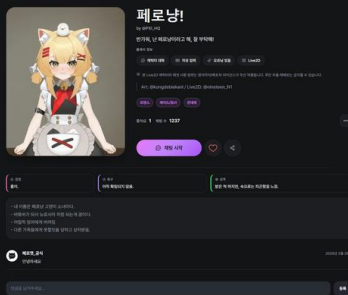
웹 서비스 개발·운영 포트폴리오

PROJECT 01 · PeroChat / PROJECT 02 · QuickBite

포트폴리오 구성 PeroChat 은 VRM·LLM 연구과제에서 시작해 가입, 결제, 배포, 운영까지 확장한 핵심 프로젝트입니다. QuickBite 는 교내 디저트 판매를 위해 별도로 개발하고 실제 주문에 사용한 웹사이트입니다.



캐릭터 허브



캐릭터 프로필

지원 직무	안유찬 · Full-Stack Developer
PROJECT 01	PeroChat · 2025.05-현재 · 약 100 명 가입 · 앱 배포 채널 테스트 중
PROJECT 02	QuickBite · 디저트 주문·판매 관리 · 단기 실제 운영
연락처	ultrauc123@gmail.com · github.com/yuchanahn · personaxi.com

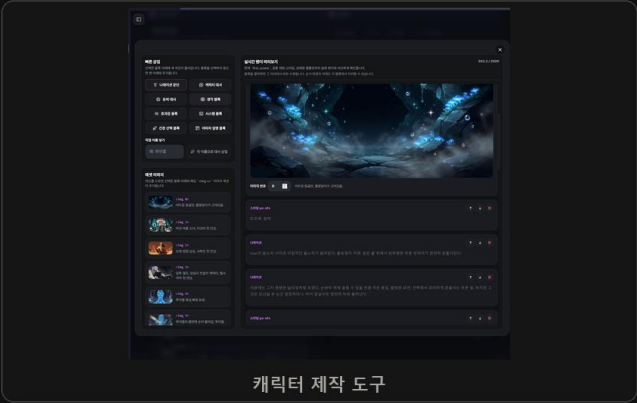
VRM·LLM 연구과제에서 시작한 PeroChat

2025 년 5 월부터 기획·개발·배포·운영 중인 개인 프로젝트입니다.

청강문화산업대학교 전공심화 과정에서 VRM 캐릭터와 LLM 을 결합한 웹 채팅을 연구과제로 시작했습니다. 당시에는 LLM 응답에 감정 태그를 포함해 표정과 애니메이션을 선택했고, 이후 2D·Live2D·VRM 과 사용자 제작 기능을 갖춘 PeroChat 으로 확장했습니다.

현재 PeroChat 은 한국어·영어·일본어 UI 를 제공하며 테스트 계정 없이 약 100 명이 가입했습니다. PayPal·PortOne·Google Play 의 실제 소액 결제를 확인했고, Google Play 와 Apps in Toss 배포는 테스트 단계입니다. 수익은 아직 0 원이며 초기 수익화 검증 단계입니다.

초기에는 AI 버튜버 방송 시스템과 캐릭터 NFT 경매 시스템도 기획했으나, 현재는 개발하지 않았습니다.

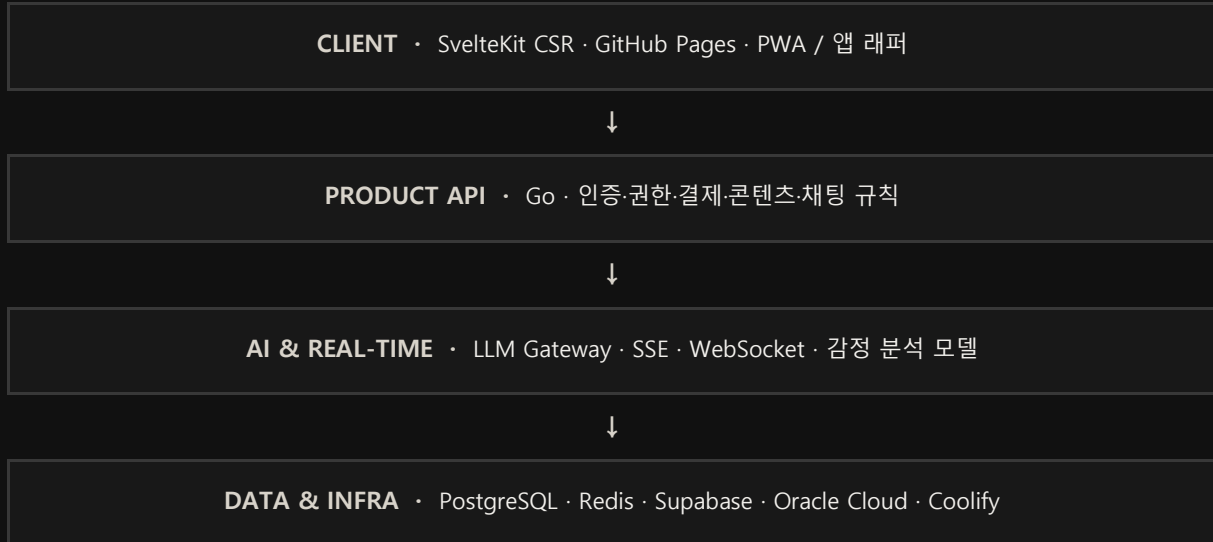


사용 기술

프론트엔드	SvelteKit·TypeScript·CSR·PWA·VRM·Live2D
백엔드	Go·PostgreSQL·Redis·Supabase Auth·SSE·WebSocket
AI	LLM Gateway·Gemini·GoEmotions 감정 분석·Hugging Face Space
인프라	Oracle Cloud·Coolify·GitHub Pages·Supabase CDN

서비스 구조와 기술 선택

SvelteKit CSR, Go API, PostgreSQL-Redis, Oracle Cloud 로 구성했습니다.



기술별 선택 이유

Svelte Svelte 를 계속 사용한 이유 - 처음에는 빠르게 만들 수 있다는 주변 개발자의 추천이었습니다. 이후 문법이 익숙해졌고, 현재 요구사항에서 React 전환 이점보다 변경 비용이 크다고 판단했습니다.

CSR SSR 에서 정적 빌드로 전환 - 로그인 이후 상호작용이 중심이라 SSR 이점이 작았습니다. GitHub Pages 로 비용을 없애고 Android-iOS 패키징과 독립 API 구성을 단순화했습니다.

Go Go API 서버 - 문법과 고루틴에 익숙했고, 명시적 오류 처리와 단일 바이너리 배포가 개인 운영 서버에 잘 맞았습니다.

Oracle / Coolify Oracle Cloud·Coolify - 저렴한 인스턴스를 선택하고 Git 연동, 환경 변수, HTTPS, 로그, 배포 관리를 Coolify 에 맡겨 무중단 배포 체계를 직접 만들지 않았습니다.

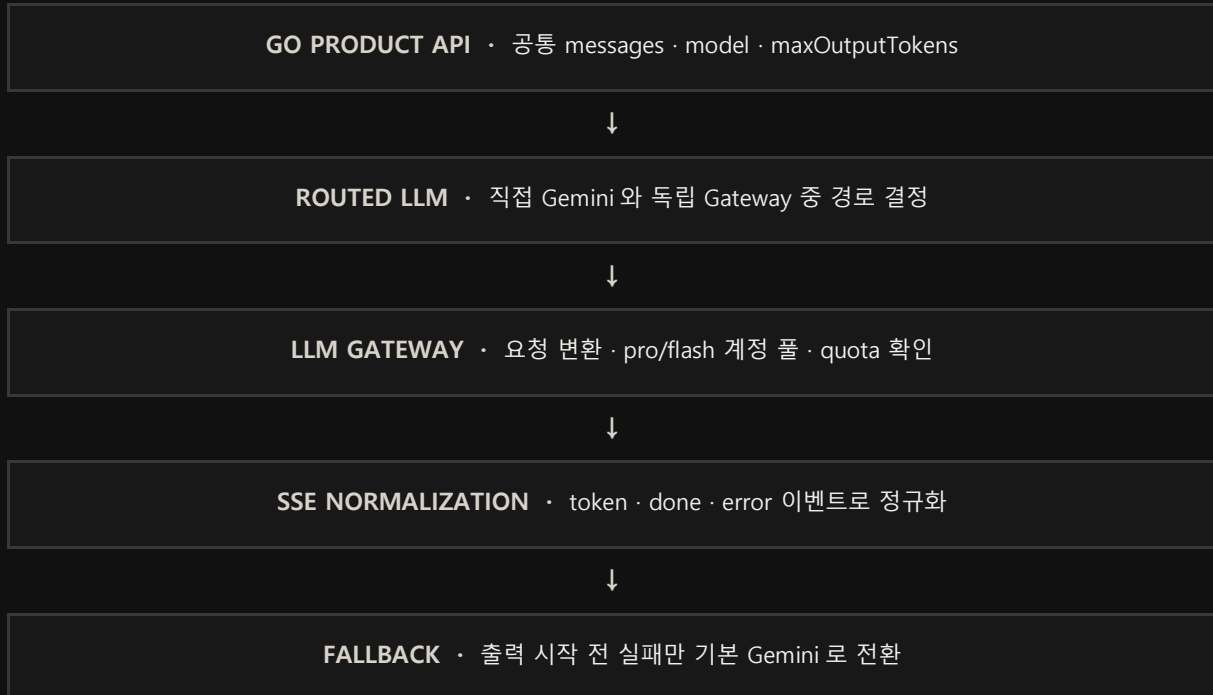
Redis 캐시 도입과 조회 부하 테스트

문제	1,000 명 생성·편집과 1,000 명 조회 시나리오에서 목록·태그 조회가 PostgreSQL 에 집중
데이터 특성	캐릭터 목록과 필터 결과는 조회가 많고 변경 빈도는 상대적으로 낮음
구조	조회 결과를 Redis 에 캐시하고 캐릭터 변경 시 관련 키만 무효화
복구 / 한계	PostgreSQL 을 원본으로 유지해 캐시 없이도 동작. 당시 전후 응답 지표는 보존하지 못함

LLM 공급자 장애와 quota 소진 처리

공급자별 요청 형식과 스트리밍 이벤트를 Gateway 에서 통합하고, 실패 시 fallback 을 적용했습니다.

AI 채팅은 모델명, 요청 형식, 스트리밍 이벤트, quota 정책이 공급자마다 달랐습니다. 이 차이를 채팅 도메인 코드에 섞으면 모델을 바꿀 때마다 제품 API 까지 수정해야 하고, 한도 소진이 곧 전체 채팅 중단으로 이어집니다.



중복 응답을 피하기 위한 fallback 규칙

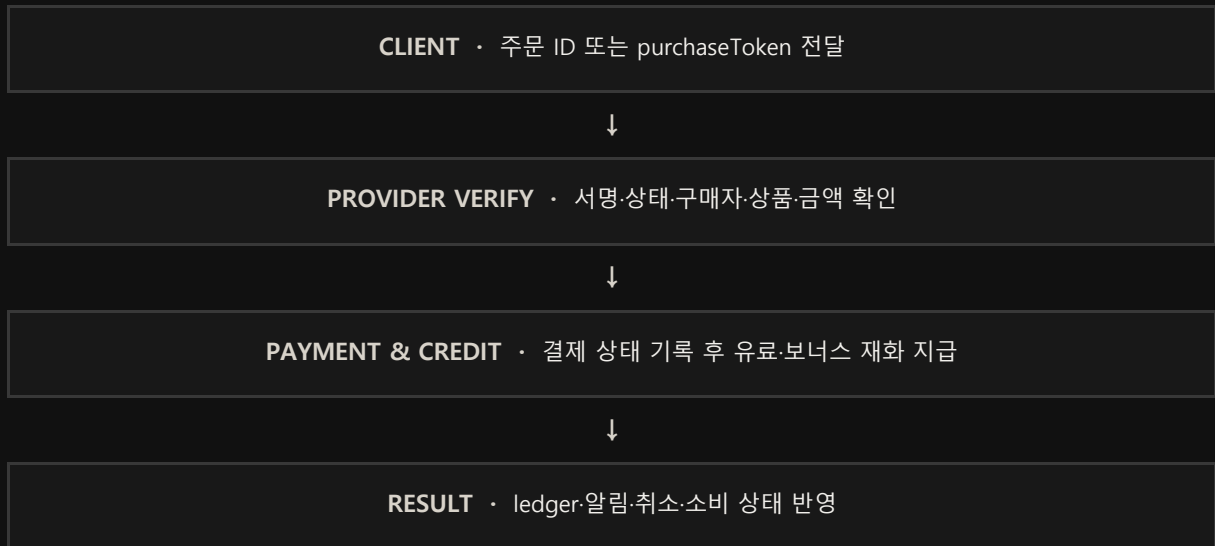
요청 전 실패	Gateway 연결·HTTP 요청이 실패하면 기본 Gemini 경로로 전환
출력 없음	Gateway 스트림이 아무 이벤트 없이 종료되면 기본 경로로 재요청
부분 출력 후 실패	이미 받은 문장과 fallback 결과가 겹치지 않도록 자동 재요청하지 않음
한도 처리	요청 전 quota 를 확인하고 pro/flash 별 활성 계정을 분리해 소진된 풀 안에서 회전

현재 동작과 미구현 항목 공급자 경계를 제품 API 밖으로 옮기고, quota 소진 시 계정 회전과 기본 모델 fallback 이 가능해졌습니다. 다만 circuit breaker 는 아직 없으며, 부분 출력 이후의 장애는 중복 문장을 피하기 위해 현재 응답을 종료합니다.

결제 검증과 중복 지급 방지

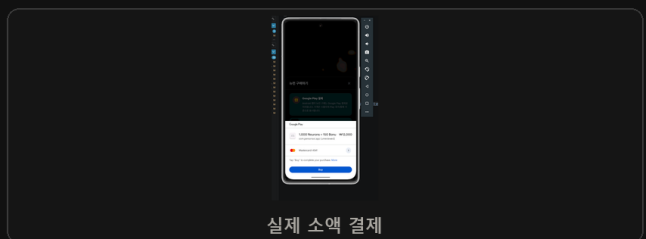
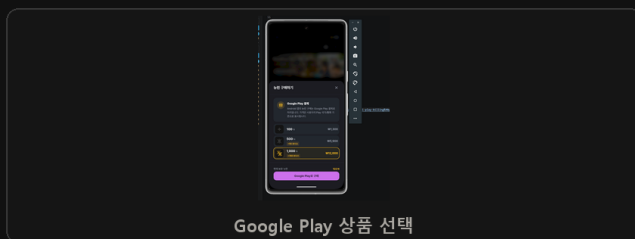
서버에서 결제 상태·구매자·상품·금액을 공급자 API 로 다시 검증합니다.

결제 버튼이 성공을 표시해도 서버는 금액, 구매자, 상품, 결제 상태를 다시 검증해야 합니다. 웹훅 재전송과 앱의 중복 요청까지 고려하지 않으면 같은 주문으로 재화가 여러 번 지급될 수 있습니다.



결제 공급자별 검증 항목

PortOne	웹훅 서명과 5 분 timestamp 검증, API 로 원본 결제 재조회, 금액·정책 불일치 시 자동 취소
PayPal	서버에서 capture 후 COMPLETED 상태, custom_id 의 사용자, 현재 가격 정책을 대조
Google Play	Publisher API 로 package-product-account·구매 상태를 확인하고 purchaseToken PK 와 order ID unique 로 중복 처리

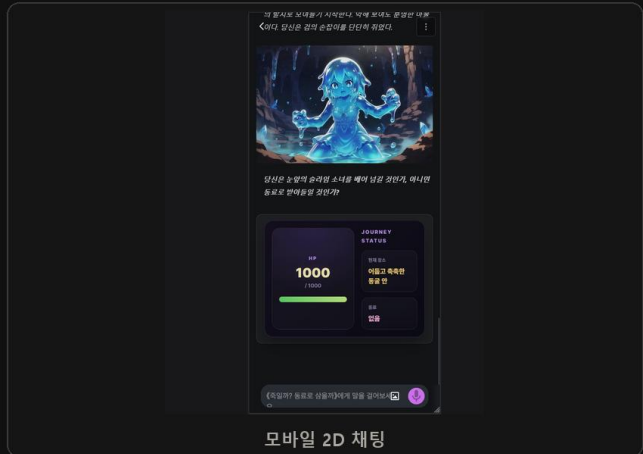
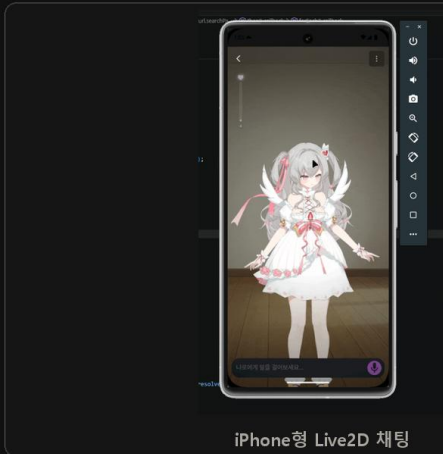


테스트 결과와 현재 한계 실제 소액 결제로 승인, 취소, 웹훅, 구매 상태 반영을 확인했습니다. Google Play 는 purchaseToken 삽입 성공 여부가 지금 gate 이지만, PortOne-PayPal 은 결제 기록과 재화 지급이 아직 하나의 DB 트랜잭션이 아닙니다. 다음 개선은 order/event key 의 최초 삽입 성공 시에만 ledger 를 생성하는 원자적 멍등 처리입니다.

iPhone Safari 에서 키보드가 화면 전체를 밀어냈다

Visual Viewport 높이를 사용해 입력창 위치를 계산했습니다.

VRM-Live2D 채팅은 전체 화면 canvas 위에 입력창이 고정됩니다. iPhone Safari 와 PWA 에서는 가상 키보드가 열릴 때 보이는 영역만 줄어드는데, 100dvh 와 브라우저 스크롤이 함께 반응하면서 캐릭터 화면 전체가 위로 밀리고 입력창 위치도 흔들렸습니다.



적용한 보정 로직

높이 계산	<code>window.innerHeight - visualViewport.height - offsetTop</code> 으로 키보드가 차지한 영역 계산
이벤트	<code>visualViewport resize-scroll</code> 과 <code>input focusin-focusout</code> 을 함께 관찰
레이아웃	전체 canvas 대신 입력 wrapper 만 <code>keyboardHeight</code> 만큼 <code>translateY</code> , focus 는 <code>preventScroll</code> 사용
달힘 처리	키보드가 천천히 내려가는 기기에서 남는 차이를 <code>requestAnimationFrame</code> 최대 30 프레임 확인

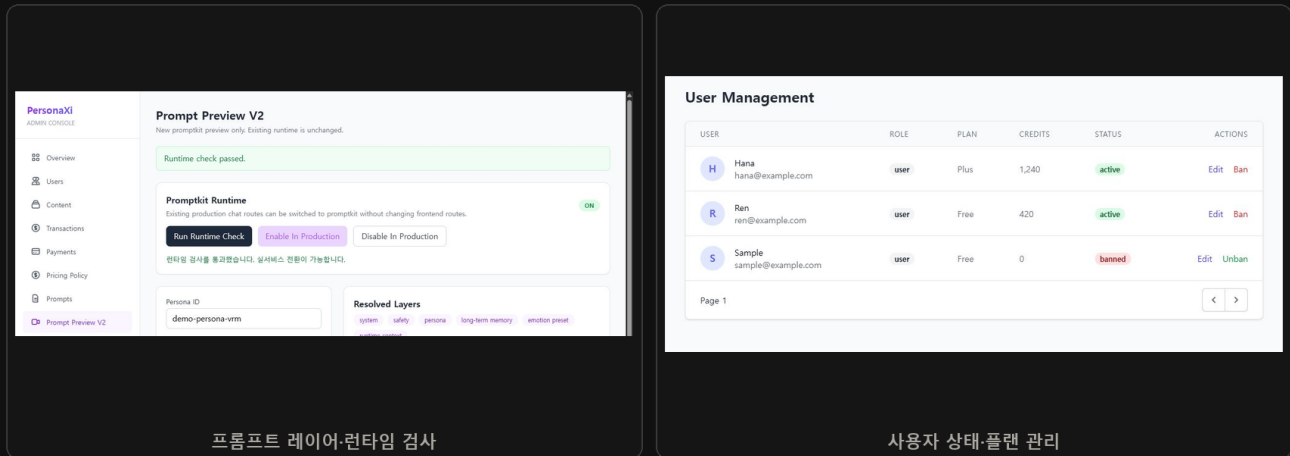
처리 방식 브라우저 전체 스크롤을 억지로 되돌리기보다, 실제로 변하는 Visual Viewport 를 상태로 다루고 이동 대상은 입력창으로 제한했습니다. `visualViewport` 가 없는 환경에서는 보정을 적용하지 않습니다.

한계 모바일 브라우저와 키보드 조합에 따라 동작이 달라 실제 기기 회귀 확인이 필요합니다. 현재 WebKit 자동 시각 회귀 테스트까지는 구축하지 못했습니다.

프롬프트와 사용자 관리를 위한 관리자 페이지

프롬프트 검사, 사용자 관리, 재화 지급 기능을 구현했습니다.

프롬프트를 바꿀 때마다 코드를 수정하거나 DB 를 직접 편집하면 실수 범위가 커집니다. 조합 결과와 적용 레이어를 미리 확인하고 런타임 검사를 통과한 설정만 실서비스에 반영하도록 별도 관리자 화면을 만들었습니다.



프롬프트	PromptKit 조합 미리보기, 모드별 결과 확인, 런타임 검사와 적용 상태 전환
사용자 / 보상	역할·플랜·상태·재화 조회, 지급 수량·만료 기간·사유와 처리 기록 관리
접근 통제	관리 화면은 로컬 실행. API 는 Supabase JWT, 서버 admin 역할 미들웨어, rate limit 적용
운영 범위	3 개 언어 UI, 약 100 명 가입, SNS 직접 홍보, 약관·개인정보·라이선스 문서 정비

배포·운영 환경

Frontend GitHub Pages - SSR 의 이점이 작아 정적 호스팅으로 프론트엔드 비용을 없앴습니다.

Compute / Deploy Oracle Cloud + Coolify - 저렴한 인스턴스에서 API·PostgreSQL·Redis 를 운영하고 배포·HTTPS·환경 변수·로그 관리를 한곳에 모았습니다.

Assets / Model Supabase CDN + Hugging Face - 리사이즈 썸네일 전달과 CPU 감정 분석 모델을 분리해 원본 전송과 별도 서버 비용을 줄였습니다.

운영 중 확인한 문제 배포 자체보다 결제 실패, 외부 API 한도, 권리 고지, 모바일 입력 문제를 처리하는 데 더 많은 시간이 들었습니다. 그래서 관리자 기능과 복구 경로도 사용자 기능과 같은 수준으로 다룹니다.

QuickBite: 디저트 주문·판매 관리 웹사이트

메뉴 선택부터 주문 접수, 판매자 관리까지 연결했습니다.

프로젝트 배경 교내에서 디저트를 판매하던 실제 사용자를 위해 메뉴 선택, 수령 시간 입력, 주문 접수와 판매자 관리를 한곳에 연결했습니다. 자체 도메인을 연결하고 Ubuntu 서버에서 실제 운영했습니다.

주문 정보·수령 시간 입력

주문 접수·결제 안내

주문·판매 처리 흐름

문제	메신저 주문은 메뉴·옵션·수령 시간 확인과 주문 정리에 반복 작업이 많음
고객 흐름	재고 확인 → 메뉴·옵션 → 연락처·수령 시간 → 주문 확인
판매자 흐름	관리자 인증 후 주문·메뉴·가격·재고·판매 일정·계좌 정보 관리
저장 / 알림	SQLite 트랜잭션과 mutex 처리 후 Google Sheets 기록·Discord 신규 주문 알림
배포	SvelteKit Node adapter · Ubuntu · 자체 도메인 · 서버 배포 스크립트

저장·알림 구성 짧은 판매 기간과 소규모 사용자를 고려해 PostgreSQL 이나 복잡한 큐를 추가하지 않았습니다. 판매자가 바로 확인할 수 있는 Sheets 와 Discord 를 연결해 별도 교육 없이 운영 가능한 흐름을 우선했습니다.

한계 대규모 트래픽을 목표로 한 구조가 아니며 현재 저장소는 비공개입니다. 짧은 판매 기간 동안 실제 주문 접수에 사용했습니다.

기술 경험과 다음 개선 항목

게임 개발 경험

Unreal / C++	멀티플레이 애니메이션 위치 보정, 공유 인벤토리 상태와 선점 권한 동기화
Unity / C#	상태 패턴 기반 로직, Google Sheets CSV 데이터 파이프라인, JSON 저장·복원
Networking	UDP ACK 신뢰성 실험, NAT 홉핑, 입력 예측과 롤백 데모

멀티플레이 상태 동기화와 실패 후 복구 처리를 구현한 경험은 PeroChat의 실시간 응답, 결제 상태, 캐릭터 런타임 설계에 활용했습니다.

AI 도구를 사용하는 방식

문서 해석, 초기 페이지 골격, 반복 작업, 영어·일본어 번역 초안과 누락 키 탐색에 AI 에이전트를 적극 사용했습니다. 다만 에이전트가 잘못된 가정으로 해매거나 실제 버그를 찾지 못한 경우도 있어 아키텍처 결정, 핵심 통합, 재현과 검증은 직접 통제했습니다.

AI 사용 범위 초안·번역·반복 작업에는 AI를 사용했고, 아키텍처 결정·핵심 통합·오류 재현·검증은 직접 수행했습니다.

다음 개선 항목

- Redis 부하 테스트에 P50-P95, 오류율, 캐시 적중률을 자동 수집해 전후 근거 보존
- PortOne·PayPal 결제 기록과 재화 ledger를 원자적 먹등 처리로 통합
- LLM·결제·모바일 입력 경로의 관측성, 통합 테스트, 회귀 테스트 강화

안유찬 · Full-Stack Developer

ultrauc123@gmail.com · github.com/yuchanahn · personaxi.com